Digital Filters

Dr Mohammed Zaki Ahmed

Contents

Digital Filters and Linear Filters

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).

Introduction

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).
- The input to the filter is a sequence of samples, and the output from the filter is also a sequence of samples.

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).
- The input to the filter is a sequence of samples, and the output from the filter is also a sequence of samples.
- So anything that is done to a sequence of data is a "Digital Filter".

Introduction

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).
- The input to the filter is a sequence of samples, and the output from the filter is also a sequence of samples.
- So anything that is done to a sequence of data is a "Digital Filter".
- The filter can be realised using shift registers or assembly code or C code,

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).
- The input to the filter is a sequence of samples, and the output from the filter is also a sequence of samples.
- So anything that is done to a sequence of data is a "Digital Filter".
- The filter can be realised using shift registers or assembly code or C code,

Introduction

- A Digital Filter is an algorithm or process that converts one digital sequence (called the input $x(n)$) to another (called the output $y(n)$).
- The input to the filter is a sequence of samples, and the output from the filter is also a sequence of samples.
- So anything that is done to a sequence of data is a "Digital Filter".
- The filter can be realised using shift registers or assembly code or C code,

  Input, x_n —— $\boxed{\text{Digital Filter}}$ —— Output, y_n

- The most commonly used filters are Linear Filter. These have a linear relationship (straight line through the origin) between input and output **sequences**.

Advantages

Digital filters offer many gains over analogue filters, some are listed below.

1. Very difficult to realise analogue filters with low cut-off frequencies.
2. Easy to change characteristics of filter.
3. No ageing/degradation or temperature dependencies.
4. Can be VERY sophisticated, and not too expensive.

Disadvantages

1. Require fast digital processors and processing.
2. The processors require power in order to complete their computation. (Digital Filters have no passive components)

Example 1

- Consider the sequence of numbers $\{1\ 2\ 3\ 1\ 2\}$,

Example 1

- Consider the sequence of numbers $\{\texttt{1 2 3 1 2}\}$,
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);

Example 1

- Consider the sequence of numbers $\{1\ \ 2\ \ 3\ \ 1\ \ 2\}$,
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);
- we obtain a sequence $\{-1\ \ -1\ \ 2\ \ -1\}$. This new sequence is obtained by doing $\{1-2\ \ 2-3\ \ 3-1\ \ 1-2\}$.

Example 1

- Consider the sequence of numbers {1 2 3 1 2},
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);
- we obtain a sequence {-1 -1 2 -1}. This new sequence is obtained by doing {1-2 2-3 3-1 1-2}.

This is Digital Filtering.

Example 1

- Consider the sequence of numbers $\{1\ \ 2\ \ 3\ \ 1\ \ 2\}$,
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);
- we obtain a sequence $\{-1\ \ -1\ \ 2\ \ -1\}$. This new sequence is obtained by doing $\{1-2\ \ 2-3\ \ 3-1\ \ 1-2\}$.

This is Digital Filtering. The filter is described using the following notation : the current number is x_n, and the previous one is x_{n-1}.

Example 1

- Consider the sequence of numbers {`1 2 3 1 2`},
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);
- we obtain a sequence {`-1 -1 2 -1`}. This new sequence is obtained by doing {`1-2 2-3 3-1 1-2`}.

This is Digital Filtering. The filter is described using the following notation : the current number is x_n, and the previous one is x_{n-1}. If a number x_{n-1} is subtracted from an adjacent number x_n;

Example 1

- Consider the sequence of numbers {1 2 3 1 2},
- if we specify a digital filter as subtracting adjacent samples from each other (in this case we subtract the number on the right from the number on the left);
- we obtain a sequence {-1 -1 2 -1}. This new sequence is obtained by doing {1-2 2-3 3-1 1-2}.

This is Digital Filtering. The filter is described using the following notation : the current number is x_n, and the previous one is x_{n-1}. If a number x_{n-1} is subtracted from an adjacent number x_n;mathematically the output y_n can be written as

$$y_n = x_n - x_{n-1}$$

Example 1

- This addition, subtraction and scaling of adjacent numbers in a sampled sequences results in scaling of the frequency spectrum.

Example 1

- This addition, subtraction and scaling of adjacent numbers in a sampled sequences results in scaling of the frequency spectrum.
- Certain frequencies will be enhanced, some unaltered and others suppressed.
- The name "Digital Filter" refers to this selection within the frequency spectrum (i.e. filtering certain frequencies).

Example 1

- This addition, subtraction and scaling of adjacent numbers in a sampled sequences results in scaling of the frequency spectrum.
- Certain frequencies will be enhanced, some unaltered and others suppressed.
- The name "Digital Filter" refers to this selection within the frequency spectrum (i.e. filtering certain frequencies).

Digital Flters are best analysed using the $\mathbb{Z}$ Transform.

$\mathbb{Z}$ Transform

Introduction

- The $\mathbb{Z}$ Transform of a sequence $x(n)$ is defined as

$$\mathbb{Z}\{x(n)\} = \sum_{n=0}^{\infty} x(n) Z^{-n}$$

where Z is a complex number.

Introduction

- The $\mathbb{Z}$ Transform of a sequence $x(n)$ is defined as

$$\mathbb{Z}\{x(n)\} = \sum_{n=0}^{\infty} x(n) Z^{-n}$$

  where Z is a complex number.
- The transformation is simply done by replacing x_{n-i} with $X \cdot Z^{-i}$ where i is any integer and X is the representation of x in the $\mathbb{Z}$ Domain. Remember $Z^0 = 1$.

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

This equation can be transformed into the $\mathbb{Z}$ Domain to give

$$Y \cdot Z^0$$

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

This equation can be transformed into the $\mathbb{Z}$ Domain to give

$$Y \cdot Z^0 \quad = \quad X \cdot Z^0$$

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

This equation can be transformed into the $\mathbb{Z}$ Domain to give

$$Y \cdot Z^0 \;=\; X \cdot Z^0 - X \cdot Z^{-1}$$

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

This equation can be transformed into the $\mathbb{Z}$ Domain to give

$$\begin{aligned} Y \cdot Z^0 &= X \cdot Z^0 - X \cdot Z^{-1} \\ Y &= X - X \cdot Z^{-1} \end{aligned}$$

Example 1 (again)

Starting with

$$y_n = x_n - x_{n-1}$$

This equation can be transformed into the $\mathbb{Z}$ Domain to give

$$\begin{aligned} Y \cdot Z^0 &= X \cdot Z^0 - X \cdot Z^{-1} \\ Y &= X - X \cdot Z^{-1} \\ Y &= X(1 - Z^{-1}) \end{aligned}$$

This results in a description between the input X and the output Y in the $\mathbb{Z}$ Domain.

$\mathbb{Z}$ Domain

- The $\mathbb{Z}$ domain is a complex number domain.

$\mathbb{Z}$ Domain

- The $\mathbb{Z}$ domain is a complex number domain.
- Z is just a complex number; and the description between input and output evaluates to a complex number.

$\mathbb{Z}$ Domain

- The $\mathbb{Z}$ domain is a complex number domain.
- Z is just a complex number; and the description between input and output evaluates to a complex number.
- Of interest are all possible values of Z that lie on the circle of radius 1.

These points are illustrated in the following examples.

Example 2

Suppose a digital filter performs

$$y_n = -x_n + 2x_{n-1} - x_{n-2}$$

Example 2

Suppose a digital filter performs

$$y_n = -x_n + 2x_{n-1} - x_{n-2}$$

The $\mathbb{Z}$ Transform of this is then

$$Y = -X + 2XZ^{-1} - XZ^{-2}$$

Example 2

Suppose a digital filter performs

$$y_n = -x_n + 2x_{n-1} - x_{n-2}$$

The $\mathbb{Z}$ Transform of this is then

$$Y = -X + 2XZ^{-1} - XZ^{-2}$$

The ratio of $\frac{Y}{X}$ is then

$$-1 + 2Z^{-1} - Z^{-2}$$

Example 3

Suppose a digital filter performs

$$y_n = x_n + 2x_{n-1} + 3x_{n-2} + 2x_{n-3} + x_{n-4}$$

Example 3

Suppose a digital filter performs

$$y_n = x_n + 2x_{n-1} + 3x_{n-2} + 2x_{n-3} + x_{n-4}$$

The $\mathbb{Z}$ Transform of this is then

$$Y =$$

Example 3

Suppose a digital filter performs

$$y_n = x_n + 2x_{n-1} + 3x_{n-2} + 2x_{n-3} + x_{n-4}$$

The $\mathbb{Z}$ Transform of this is then

$$Y = X(1 + 2Z^{-1} + 3Z^{-2} + 2Z^{-3} + Z^{-4})$$

Example 3

Suppose a digital filter performs

$$y_n = x_n + 2x_{n-1} + 3x_{n-2} + 2x_{n-3} + x_{n-4}$$

The $\mathbb{Z}$ Transform of this is then

$$Y = X(1 + 2Z^{-1} + 3Z^{-2} + 2Z^{-3} + Z^{-4})$$

The ratio of $\frac{Y}{X}$ is then

$$1 + 2Z^{-1} + 3Z^{-2} + 2Z^{-3} + Z^{-4}$$

Impulse Response

A Digital Impulse is a sequence that contains a single unity element followed by zeros. The impulse response is the response of a filter (the output from the filter) if the input was an impulse.

Input, $x_n = [1, 0, 0, 0, 0, 0, \cdots]$ — $\boxed{\text{Digital Filter}}$ — Output, y_n

A Digital Impulse is a sequence that contains a single unity element followed by zeros. The impulse response is the response of a filter (the output from the filter) if the input was an impulse.

Input, $x_n = [1, 0, 0, 0, 0, 0, \cdots]$ —— Digital Filter —— Output, y_n

- If the impulse response of a filter results in an output that has a finite number of non-zero outputs the filter is called a Finite Impulse Response (FIR) filter.

A Digital Impulse is a sequence that contains a single unity element followed by zeros. The impulse response is the response of a filter (the output from the filter) if the input was an impulse.

$$\text{Input, } x_n = [1, 0, 0, 0, 0, 0, \cdots] \longrightarrow \boxed{\text{Digital Filter}} \longrightarrow \text{Output, } y_n$$

- If the impulse response of a filter results in an output that has a finite number of non-zero outputs the filter is called a Finite Impulse Response (FIR) filter. Thus if the sequence y_n in the figure above is a finite length sequence; the filter is a FIR filter

A Digital Impulse is a sequence that contains a single unity element followed by zeros. The impulse response is the response of a filter (the output from the filter) if the input was an impulse.

Input, $x_n = [1, 0, 0, 0, 0, 0, \cdots]$ —— $\boxed{\text{Digital Filter}}$ —— Output, y_n

- If the impulse response of a filter results in an output that has a finite number of non-zero outputs the filter is called a Finite Impulse Response (FIR) filter. Thus if the sequence y_n in the figure above is a finite length sequence; the filter is a FIR filter
- Filters that create an infinite number of non-zero outputs and are called Infinite Impulse Response Filters (IIR Filters).

A filter is stable if its impulse response decays to zero as time (or the number of samples) increases.

Stability

A filter is stable if its impulse response decays to zero as time (or the number of samples) increases.

- Any system has has a finite length impulse response will be stable.

Stability

A filter is stable if its impulse response decays to zero as time (or the number of samples) increases.

- Any system has has a finite length impulse response will be stable.
- Infinite impulse response systems can be stable,

Stability

A filter is stable if its impulse response decays to zero as time (or the number of samples) increases.

- Any system has has a finite length impulse response will be stable.
- Infinite impulse response systems can be stable, unstable (get larger and larger) or

Stability

A filter is stable if its impulse response decays to zero as time (or the number of samples) increases.

- Any system has has a finite length impulse response will be stable.
- Infinite impulse response systems can be stable, unstable (get larger and larger) or can oscillate.

The $\mathbb{Z}$ Domain relationship between input and output sequences is the transfer function of the filter.

Transfer Function

The $\mathbb{Z}$ Domain relationship between input and output sequences is the transfer function of the filter.

- This results in a complex number describing the relationship between input and output.

Transfer Function

The $\mathbb{Z}$ Domain relationship between input and output sequences is the transfer function of the filter.

- This results in a complex number describing the relationship between input and output.
- This complex number is also best viewed as having magnitude and phase.

Transfer Function

The $\mathbb{Z}$ Domain relationship between input and output sequences is the transfer function of the filter.

- This results in a complex number describing the relationship between input and output.
- This complex number is also best viewed as having magnitude and phase.
- The unit circle in the $\mathbb{Z}$ Domain is the complex number $1e^{j\theta}$

Transfer Function

The $\mathbb{Z}$ Domain relationship between input and output sequences is the transfer function of the filter.

- This results in a complex number describing the relationship between input and output.
- This complex number is also best viewed as having magnitude and phase.
- The unit circle in the $\mathbb{Z}$ Domain is the complex number $1e^{j\theta}$
- θ on the unit circle is the Fourier Transform and is equal to $2\pi\frac{f}{F_s}$, where f is the frequency and F_s is the sampling frequency.

$$Z = e^{j2\pi\frac{f}{F_s}} = e^{j\theta}$$

Also $-\pi \leq \theta \leq \pi$ or $0 \leq \theta \leq 2\pi$.

Magnitude Response On Unit Circle

- This describes the relationship between the magnitude of the output to the magnitude of the input.
- Filters can amplify, attenuate or leave unchanged the magnitude of the input depending on the value of Z.

Phase Response On Unit Circle

- The Phase Response represents the phase offset between the input and output from the filter.
- Filters that operate with real numbers must have phase responses that are anti–symmetric about $\theta = 0$.

Phase Response On Unit Circle

- The Phase Response represents the phase offset between the input and output from the filter.
- Filters that operate with real numbers must have phase responses that are anti–symmetric about $\theta = 0$. This means for a given sampling frequency F_s, the phase response at a frequency f must be equal to the negative value of the phase response at a frequency $-f$.

Pole and Zero

- A Pole in the $\mathbb{Z}$ Domain is a value of Z for which the magnitude response is infinite.
- A Zero in the $\mathbb{Z}$ Domain is a value of Z for which the magnitude response is zero.

The number of Pole and Zero's are always the same.

Example 4

Suppose a digital filter is specified with the following transfer function

$$\frac{Y}{X} = \frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}$$

We can evaluate the manner of combining x_n and y_n as follows

Example 4

Suppose a digital filter is specified with the following transfer function

$$\frac{Y}{X} = \frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}$$

We can evaluate the manner of combining x_n and y_n as follows

- Expanding the transfer function gives

$$\frac{Y}{X} \quad = \quad \frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}$$

Example 4

Suppose a digital filter is specified with the following transfer function

$$\frac{Y}{X} = \frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}$$

We can evaluate the manner of combining x_n and y_n as follows

- Expanding the transfer function gives

$$\begin{aligned}\frac{Y}{X} &= \frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\ &= \frac{[(Z-(0.5+j0.5))(Z-(0.5-j0.5))][(Z-(-1))]}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}\end{aligned}$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= [Z^2 - (0.5 - j0.5)Z - (0.5 + j0.5)Z + \\
&\quad (0.5 + j0.5)(0.5 - j0.5)] \times \\
&\qquad \times \frac{[(Z - (-1))]}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))}
\end{aligned}
$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= [Z^2 - (0.5 - j0.5)Z - (0.5 + j0.5)Z + \\
&\quad (0.5 + j0.5)(0.5 - j0.5)] \times \\
&\quad \times \frac{[(Z - (-1))]}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))} \\
&= [Z^2 - 0.5Z + j0.5Z - 0.5Z - j0.5Z + 0.5^2 + 0.5^2] \times \\
&\quad \times \frac{(Z + 1)}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))}
\end{aligned}
$$

Example 4

$$\begin{aligned}
\frac{Y}{X} &= [Z^2-(0.5-j0.5)Z-(0.5+j0.5)Z+\\
&\quad (0.5+j0.5)(0.5-j0.5)]\times\\
&\qquad \times\frac{[(Z-(-1))]}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}\\
&= [Z^2-0.5Z+j0.5Z-0.5Z-j0.5Z+0.5^2+0.5^2]\times\\
&\qquad \times\frac{(Z+1)}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}\\
&= \frac{[Z^2-Z+0.5][(Z+1)]}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}
\end{aligned}$$

Example 4

$$\frac{Y}{X} = \frac{Z^3 - Z^2 + 0.5Z + Z^2 - Z + 0.5}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))}$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= \frac{Z^3 - Z^2 + 0.5Z + Z^2 - Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))}
\end{aligned}
$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= \frac{Z^3 - Z^2 + 0.5Z + Z^2 - Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z^2 + (j0.9)Z - (j0.9)Z + 0.9^2)(Z-(-0.9))}
\end{aligned}
$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= \frac{Z^3 - Z^2 + 0.5Z + Z^2 - Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z-(j0.9))(Z-(-j0.9))(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z^2+(j0.9)Z-(j0.9)Z+0.9^2)(Z-(-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z^2+0.81)(Z+0.9)}
\end{aligned}
$$

Example 4

$$
\begin{aligned}
\frac{Y}{X} &= \frac{Z^3 - Z^2 + 0.5Z + Z^2 - Z + 0.5}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z - (j0.9))(Z - (-j0.9))(Z - (-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z^2 + (j0.9)Z - (j0.9)Z + 0.9^2)(Z - (-0.9))} \\
&= \frac{Z^3 - 0.5Z + 0.5}{(Z^2 + 0.81)(Z + 0.9)} \\
&= \frac{Z^3 - 0.5Z + 0.5}{Z^3 + 0.9Z^2 + 0.81Z + 0.729}
\end{aligned}
$$

Example 4

- Rewriting this using only negative powers of Z,

$$\frac{Y}{X} = \frac{Z^3 - 0.5Z + 0.5}{Z^3 + 0.9Z^2 + 0.81Z + 0.729}$$

Example 4

- Rewriting this using only negative powers of Z,

$$\frac{Y}{X} = \frac{Z^3 - 0.5Z + 0.5}{Z^3 + 0.9Z^2 + 0.81Z + 0.729}$$
$$= \frac{(Z^3 - 0.5Z + 0.5)/Z^3}{(Z^3 + 0.9Z^2 + 0.81Z + 0.729)/Z^3}$$

- Rewriting this using only negative powers of Z,

$$\begin{aligned}
\frac{Y}{X} &= \frac{Z^3 - 0.5Z + 0.5}{Z^3 + 0.9Z^2 + 0.81Z + 0.729} \\
&= \frac{(Z^3 - 0.5Z + 0.5)/Z^3}{(Z^3 + 0.9Z^2 + 0.81Z + 0.729)/Z^3} \\
&= \frac{1 - 0.5Z^{-2} + 0.5Z^{-3}}{1 + 0.9Z^{-1} + 0.81Z^{-2} + 0.729Z^{-3}}
\end{aligned}$$

Example 4

$$Y(1 + 0.9Z^{-1} + 0.81Z^{-2} + 0.729Z^{-3}) = \\ X(1 - 0.5Z^{-2} + 0.5Z^{-3})$$

Example 4

$$
\begin{aligned}
Y(1 + 0.9Z^{-1} + 0.81Z^{-2} + 0.729Z^{-3}) = \\
X(1 - 0.5Z^{-2} + 0.5Z^{-3}) \\
Y + 0.9YZ^{-1} + 0.81YZ^{-2} + 0.729YZ^{-3} = \\
X - 0.5XZ^{-2} + 0.5XZ^{-3}
\end{aligned}
$$

$$Y(1 + 0.9Z^{-1} + 0.81Z^{-2} + 0.729Z^{-3}) = X(1 - 0.5Z^{-2} + 0.5Z^{-3})$$

$$Y + 0.9YZ^{-1} + 0.81YZ^{-2} + 0.729YZ^{-3} = X - 0.5XZ^{-2} + 0.5XZ^{-3}$$

$$Y = X - 0.5XZ^{-2} + 0.5XZ^{-3} - 0.9YZ^{-1} - 0.81YZ^{-2} - 0.729YZ^{-3}$$

Example 4

$$
\begin{aligned}
Y(1 + 0.9Z^{-1} + 0.81Z^{-2} + 0.729Z^{-3}) = \\
X(1 - 0.5Z^{-2} + 0.5Z^{-3}) \\
Y + 0.9YZ^{-1} + 0.81YZ^{-2} + 0.729YZ^{-3} = \\
X - 0.5XZ^{-2} + 0.5XZ^{-3}
\end{aligned}
$$

$$Y = X - 0.5XZ^{-2} + 0.5XZ^{-3} - 0.9YZ^{-1} - 0.81YZ^{-2} - 0.729YZ^{-3}$$

$$y_n = x_n - 0.5x_{n-2} + 0.5x_{n-3} - 0.9y_{n-1} - 0.81y_{n-2} - 0.729y_{n-3}$$

- The criteria of stability in the $\mathbb{Z}$ Domain is that ALL observed poles lie WITHIN the unit circle.

Stability in the $\mathbb{Z}$ Domain

- The criteria of stability in the $\mathbb{Z}$ Domain is that ALL observed poles lie WITHIN the unit circle.
- If this criteria is not met, the the implication is that the summation in the definition of the $\mathbb{Z}$ transform will become infinite.

$$\mathbb{Z}\{x(n)\} = \sum_{n=0}^{\infty} x(n)Z^{-n}$$

Stability in the $\mathbb{Z}$ Domain

- The criteria of stability in the $\mathbb{Z}$ Domain is that ALL observed poles lie WITHIN the unit circle.
- If this criteria is not met, the the implication is that the summation in the definition of the $\mathbb{Z}$ transform will become infinite.

$$\mathbb{Z}\{x(n)\} = \sum_{n=0}^{\infty} x(n) Z^{-n}$$

Note the stability of a filter cannot be predicted by looking at the magnitude and phase response of the filter on the unit circle. Only the location of the poles in the entire $\mathbb{Z}$ Domain determine the stability of the filter.

Example 5

Suppose a digital filter is specified with the following transfer function

$$\frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-1.1))}$$

Determine the relationship between y_n and x_n. Note this filter is unstable;

Example 5

Suppose a digital filter is specified with the following transfer function

$$\frac{(Z-(0.5+j0.5))(Z-(0.5-j0.5))(Z-(-1))}{(Z-(j0.9))(Z-(-j0.9))(Z-(-1.1))}$$

Determine the relationship between y_n and x_n. Note this filter is unstable; and the relationship between y_n and x_n is

$$y_n = x_n - 0.5x_{n-2} + 0.5x_{n-3} - 1.1y_{n-1} - 0.81y_{n-2} - 0.891y_{n-3}$$

FIR Filter

Introduction

- FIR Filters are filters with a finite impulse response.
- Assuming the impulse response of a filter is

$$[h_0, h_1, h_2, \cdots, h_{k-1}]$$

If the input sampled sequence to the filter is

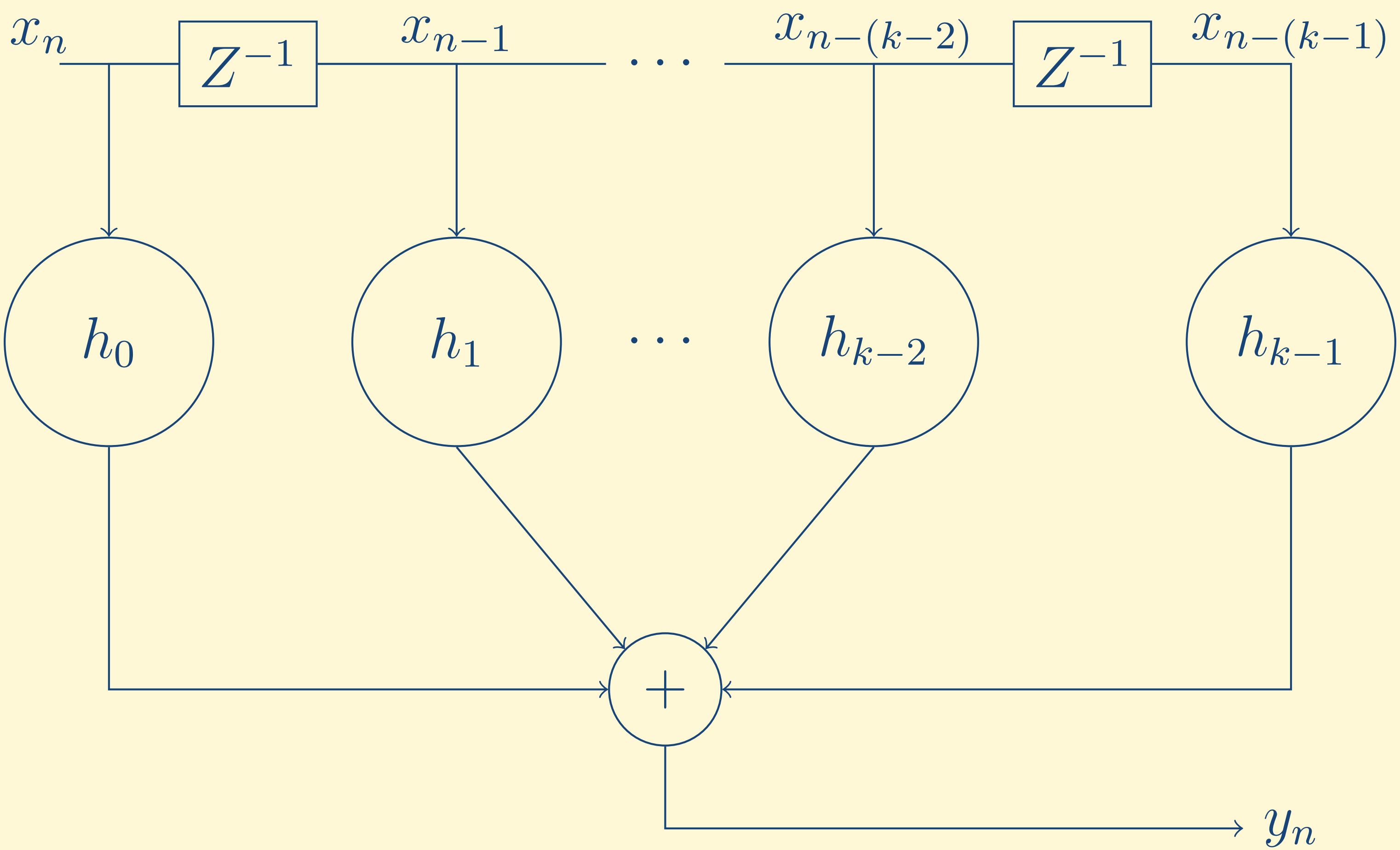
$$[x_n, x_{n-1}, x_{n-2} \cdots]$$

The output sequence is

$$y_n = x_n h_0 + x_{n-1} h_1 + \cdots + x_{n-k-1} h_{k-1}$$

The filter can be represented using a delay line as follows

FIR Notation



FIR Notation

- Remember, the sequence $h_0, h_1 \cdots h_{k-1}$ is just a sequence of k numbers stored in memory.
- The filter uses only the k most recent samples of the input data to compute the output.
  - Thus only the sequence $x_n, x_{n-1} \cdots , x_{n-(k-1)}$ is used (where x_n is the current sample).
  - Older samples are discarded.

Introduction

- The magnitude response of a filter is the magnitude of the transfer function on the unit circle on the $\mathbb{Z}$ Plane.
- It relates the magnitude of every normalised frequency $\frac{f}{F_s}$ at the output of the filter to the magnitude at the input.

Introduction

- The phase response of a filter is the phase of the transfer function on the unit circle on the $\mathbb{Z}$ Plane.
- It relates the phase offset of every normalised frequency $\frac{f}{F_s}$ at the output of the filter to the phase at the input.

Implementation

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).
- These processors have fixed point or floating point representation of numbers.

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).
- These processors have fixed point or floating point representation of numbers.
- These processors also have a finite number of bits to represent numbers.

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).
- These processors have fixed point or floating point representation of numbers.
- These processors also have a finite number of bits to represent numbers.

The effect of the number of bits and the representation of numbers on digital filters needs to be taken into account for efficient filter implementation.

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).
- These processors have fixed point or floating point representation of numbers.
- These processors also have a finite number of bits to represent numbers.

The effect of the number of bits and the representation of numbers on digital filters needs to be taken into account for efficient filter implementation. If b bits are used to represent a binary number then

Implementation

- Digital filters normally operate on microprocessors (e.g. 8051, ARM, etc).
- These processors have fixed point or floating point representation of numbers.
- These processors also have a finite number of bits to represent numbers.

The effect of the number of bits and the representation of numbers on digital filters needs to be taken into account for efficient filter implementation. If b bits are used to represent a binary number then

- there are a maximum of 2^b possible numbers that can be represented

Fixed Point Numbers

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed.

Introduction

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed. The numbers to the left of the point represent the integer part and the numbers to the right represent the fractional part.

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed.The numbers to the left of the point represent the integer part and the numbers to the right represent the fractional part.
For example in binary

$$1.1101 = 1\times 2^0 + 1\times 2^{-1} + 1\times 2^{-2} + 0\times 2^{-3} + 1\times 2^{-4} = 1.8125$$

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed.The numbers to the left of the point represent the integer part and the numbers to the right represent the fractional part.
For example in binary

$$1.1101 = 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.8125$$

If the decimal point is shifted,

$$11.101 = 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 3.625$$

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed.The numbers to the left of the point represent the integer part and the numbers to the right represent the fractional part.
For example in binary

$$1.1101 = 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.8125$$

If the decimal point is shifted,

$$11.101 = 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 3.625$$

Only positive numbers can be represented in this manner.

In a fixed point representation, it is assumed that the position of the decimal/binary point is fixed.The numbers to the left of the point represent the integer part and the numbers to the right represent the fractional part.
For example in binary

$$1.1101 = 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4} = 1.8125$$

If the decimal point is shifted,

$$11.101 = 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 3.625$$

Only positive numbers can be represented in this manner. The manner in which negative numbers are represented leads to different forms of fixed point arithmetic.

Sign–Magnitude

- The most significant bit represents the sign of the number.

Sign–Magnitude

- The most significant bit represents the sign of the number.
- Zero usually represents positive numbers and one represents negative numbers.

Sign–Magnitude

- The most significant bit represents the sign of the number.
- Zero usually represents positive numbers and one represents negative numbers. The number zero has two representations $10000\cdots$ and $00000\cdots$.

For example

$$01.101 =$$

Sign–Magnitude

- The most significant bit represents the sign of the number.
- Zero usually represents positive numbers and one represents negative numbers. The number zero has two representations $10000\cdots$ and $00000\cdots$.

For example

$$01.101 = +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625$$

Sign–Magnitude

- The most significant bit represents the sign of the number.
- Zero usually represents positive numbers and one represents negative numbers. The number zero has two representations $10000\cdots$ and $00000\cdots$.

For example

$$01.101 = +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625$$

$$11.101 =$$

Sign–Magnitude

- The most significant bit represents the sign of the number.
- Zero usually represents positive numbers and one represents negative numbers. The number zero has two representations $10000\cdots$ and $00000\cdots$.

For example

$$01.101 = +1 \times \left[1 \times 2^{0} + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625$$

$$11.101 = -1 \times \left[1 \times 2^{0} + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = -1.625$$

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number.

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number. There are two ways of representing zero (just like the Sign–Magnitude notation).

For example

$$01.101 \quad =$$

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number. There are two ways of representing zero (just like the Sign–Magnitude notation).

For example

$$01.101 \;=\; +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625$$

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number. There are two ways of representing zero (just like the Sign–Magnitude notation).

For example

$$\begin{aligned} 01.101 &= +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625 \\ 10.010 &= \end{aligned}$$

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number. There are two ways of representing zero (just like the Sign–Magnitude notation).

For example

$$\begin{aligned} 01.101 &= +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625 \\ 10.010 &= -1 \times \hat{0}.\hat{0}\hat{1}\hat{0} = \end{aligned}$$

1's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number. There are two ways of representing zero (just like the Sign–Magnitude notation).

For example

$$\begin{aligned} 01.101 &= +1 \times \left[1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}\right] = +1.625 \\ 10.010 &= -1 \times \hat{0}.\hat{0}\hat{1}\hat{0} = -1 \times 1.101 = -1.625 \end{aligned}$$

2's–Complement

- Positive numbers are identical as the Sign–Magnitude form.

2's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number and **adding** a 1 to the least significant bit.

2's–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number and **adding** a 1 to the least significant bit.
- The main advantage of 2's complement is that most operations are not dependent on the sign of the operands and furthermore are identical to operations on unsigned binary integers[1].

2’s–Complement

- Positive numbers are identical as the Sign–Magnitude form.
- Negative numbers are obtained by complementing all the bits of the positive number and **adding** a 1 to the least significant bit.
- The main advantage of 2’s complement is that most operations are not dependent on the sign of the operands and furthermore are identical to operations on unsigned binary integers[1].
- An additional advantage is that if the final sum is within the range; even if partial sums result in overflows the final sum will be correct.

Examples are shown in the next two slides

[1] See http://en.wikipedia.org/wiki/Computer_numbering_formats

Binary Number	Value	Sign-Magnitude	1's Complement	2's Complement
0.000	0.000000	+0.000000	+0.000000	+0.000000
0.001	0.125000	+0.125000	+0.125000	+0.125000
0.010	0.250000	+0.250000	+0.250000	+0.250000
0.011	0.375000	+0.375000	+0.375000	+0.375000
0.100	0.500000	+0.500000	+0.500000	+0.500000
0.101	0.625000	+0.625000	+0.625000	+0.625000
0.110	0.750000	+0.750000	+0.750000	+0.750000
0.111	0.875000	+0.875000	+0.875000	+0.875000
1.000	1.000000	-0.000000	-0.875000	-1.000000
1.001	1.125000	-0.125000	-0.750000	-0.875000
1.010	1.250000	-0.250000	-0.625000	-0.750000
1.011	1.375000	-0.375000	-0.500000	-0.625000
1.100	1.500000	-0.500000	-0.375000	-0.500000
1.101	1.625000	-0.625000	-0.250000	-0.375000
1.110	1.750000	-0.750000	-0.125000	-0.250000
1.111	1.875000	-0.875000	-0.000000	-0.125000

Binary Number	Value	Sign-Magnitude	1's Complement	2's Complement
0000.	0.000000	+0.000000	+0.000000	+0.000000
0001.	1.000000	+1.000000	+1.000000	+1.000000
0010.	2.000000	+2.000000	+2.000000	+2.000000
0011.	3.000000	+3.000000	+3.000000	+3.000000
0100.	4.000000	+4.000000	+4.000000	+4.000000
0101.	5.000000	+5.000000	+5.000000	+5.000000
0110.	6.000000	+6.000000	+6.000000	+6.000000
0111.	7.000000	+7.000000	+7.000000	+7.000000
1000.	8.000000	-0.000000	-7.000000	-8.000000
1001.	9.000000	-1.000000	-6.000000	-7.000000
1010.	10.000000	-2.000000	-5.000000	-6.000000
1011.	11.000000	-3.000000	-4.000000	-5.000000
1100.	12.000000	-4.000000	-3.000000	-4.000000
1101.	13.000000	-5.000000	-2.000000	-3.000000
1110.	14.000000	-6.000000	-1.000000	-2.000000
1111.	15.000000	-7.000000	-0.000000	-1.000000

Floating Point Numbers

Mantissa & Exponent

- Numbers are represented using two fixed point numbers a mantissa m, and an exponent e.

Mantissa & Exponent

- Numbers are represented using two fixed point numbers a mantissa m, and an exponent e.

$$\text{Number } = m \cdot 2^e$$

Note it is not essential to use the powers of 2; any powers can be used.

Mantissa & Exponent

- Numbers are represented using two fixed point numbers a mantissa m, and an exponent e.

  $$\text{Number } = m \cdot 2^e$$

  Note it is not essential to use the powers of 2; any powers can be used.
- There are many ways to create floating point numbers.

Mantissa & Exponent

- Numbers are represented using two fixed point numbers a mantissa m, and an exponent e.

  $$\text{Number } = m \cdot 2^e$$

  Note it is not essential to use the powers of 2; any powers can be used.
- There are many ways to create floating point numbers. One (common) method is keeping the mantissa in the range

  $$-1 \leq m < 1$$

  ; and keeping the exponent as an unsigned binary integer.

Mantissa & Exponent

- Numbers are represented using two fixed point numbers a mantissa m, and an exponent e.

  $$\text{Number } = m \cdot 2^e$$

  Note it is not essential to use the powers of 2; any powers can be used.
- There are many ways to create floating point numbers. One (common) method is keeping the mantissa in the range

  $$-1 \leq m < 1$$

  ; and keeping the exponent as an unsigned binary integer.
- The number of bits b must be split into two; some bits for the mantissa and some for the exponent.

Resolution

- The resolution of fixed point numbers is uniform; however for floating point numbers the resolution varies; the resolution is fine for small numbers and coarse for large numbers.

Resolution

- The resolution of fixed point numbers is uniform; however for floating point numbers the resolution varies; the resolution is fine for small numbers and coarse for large numbers.
- The greater the number of bits used for the exponent the larger the dynamic range of the numbers, but the lower the accuracy of the numbers.

Resolution

- The resolution of fixed point numbers is uniform; however for floating point numbers the resolution varies; the resolution is fine for small numbers and coarse for large numbers.
- The greater the number of bits used for the exponent the larger the dynamic range of the numbers, but the lower the accuracy of the numbers.
- Normally around $\frac{3}{4}$ of the bits are used for the mantissa and $\frac{1}{4}$ of the bits are used for the exponent[2].

Resolution

- The resolution of fixed point numbers is uniform; however for floating point numbers the resolution varies; the resolution is fine for small numbers and coarse for large numbers.
- The greater the number of bits used for the exponent the larger the dynamic range of the numbers, but the lower the accuracy of the numbers.
- Normally around $\frac{3}{4}$ of the bits are used for the mantissa and $\frac{1}{4}$ of the bits are used for the exponent[2].

Floating point representation results in rounding errors for both addition and multiplication whereas in fixed point notation, only multiplication results in rounding errors.

[2] See the IEEE 754 standard for 32 bit floating point numbers

- In all cases there are only $2^8 = 256$ possible numbers.

- In all cases there are only $2^8 = 256$ possible numbers.
- The 2's Complement number ranged from -1 to almost $+1$;

- In all cases there are only $2^8 = 256$ possible numbers.
- The 2's Complement number ranged from -1 to almost $+1$; whereas the floating point numbers range depends on the number of bits used for the exponent.

- In all cases there are only $2^8 = 256$ possible numbers.
- The 2's Complement number ranged from -1 to almost $+1$; whereas the floating point numbers range depends on the number of bits used for the exponent.

Thus when we implement digital filters we need to know the range of all the numbers. This includes the range of numbers at

1. The filter input
2. The filter output
3. The filter coefficients

- In all cases there are only $2^8 = 256$ possible numbers.
- The 2's Complement number ranged from -1 to almost $+1$; whereas the floating point numbers range depends on the number of bits used for the exponent.

Thus when we implement digital filters we need to know the range of all the numbers. This includes the range of numbers at

1. The filter input
2. The filter output
3. The filter coefficients

The addition and multiplication can be done using the instructions of the microprocessor or using new routines (written by the programmer/engineer).

Digital Filters

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients.

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients. Since these are finite precision, there are differences between the ideal poles and zeros and the actual poles and zeros of the filter.

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients. Since these are finite precision, there are differences between the ideal poles and zeros and the actual poles and zeros of the filter. Poles close to the unit circle are the most sensitive.

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients. Since these are finite precision, there are differences between the ideal poles and zeros and the actual poles and zeros of the filter. Poles close to the unit circle are the most sensitive.
- The addition and multiplication of finite precision numbers will also lead to rounding of the results.

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients. Since these are finite precision, there are differences between the ideal poles and zeros and the actual poles and zeros of the filter. Poles close to the unit circle are the most sensitive.
- The addition and multiplication of finite precision numbers will also lead to rounding of the results.
- Also IIR filters (filters with feedback) are more sensitive. The feedback of rounding errors can cause oscillations in computation.

Finite Precision Effects

- The size of the registers where filter coefficients and data stored determines the accuracy with which we can represent filter coefficients. Since these are finite precision, there are differences between the ideal poles and zeros and the actual poles and zeros of the filter. Poles close to the unit circle are the most sensitive.
- The addition and multiplication of finite precision numbers will also lead to rounding of the results.
- Also IIR filters (filters with feedback) are more sensitive. The feedback of rounding errors can cause oscillations in computation. **This effect is known as Limit Cycle**.

Pole Zero Sensitivity

The effect of filter finite precision coefficient on the poles (and zeros) of a filter depends on how close the poles are to each other

Pole Zero Sensitivity

The effect of filter finite precision coefficient on the poles (and zeros) of a filter depends on how close the poles are to each other

- Poles that are close together are more sensitive to finite precision effects

Thus a strategy for implementing filters is to

Pole Zero Sensitivity

The effect of filter finite precision coefficient on the poles (and zeros) of a filter depends on how close the poles are to each other

- Poles that are close together are more sensitive to finite precision effects

Thus a strategy for implementing filters is to

- Cascade many smaller filters which have poles (and zeros) as far apart as possible.

Pole Zero Sensitivity

The effect of filter finite precision coefficient on the poles (and zeros) of a filter depends on how close the poles are to each other

- Poles that are close together are more sensitive to finite precision effects

Thus a strategy for implementing filters is to

- Cascade many smaller filters which have poles (and zeros) as far apart as possible.
- Have in parallel many smaller filters with the same criteria above.

In practice the cascade form is used as it is more robust to quantisation effects. Also poles closer to the origin are implemented first.
Typically filters with two or fewer poles are implemented.

Strategy

If the transfer function has m poles and zeros, with zeros at locations α and poles at locations β; the transfer function can be written as

$$H(Z) = \frac{(Z-\alpha_0)(Z-\alpha_1)\cdots(Z-\alpha_{m-1})}{(Z-\beta_0)(Z-\beta_1)\cdots(Z-\beta_{m-1})}$$

The design choice for practical implementation is the cascade form

Strategy

If the transfer function has m poles and zeros, with zeros at locations α and poles at locations β; the transfer function can be written as

$$H(Z) = \frac{(Z-\alpha_0)(Z-\alpha_1)\cdots(Z-\alpha_{m-1})}{(Z-\beta_0)(Z-\beta_1)\cdots(Z-\beta_{m-1})}$$

The design choice for practical implementation is the cascade form

$$H(Z) = \prod \frac{b_{k0} + b_{k1}Z^{-1} + b_{k2}Z^{-2}}{1 + a_{k1}Z^{-1} + a_{k2}Z^{-2}}$$

If the transfer function has m poles and zeros, with zeros at locations α and poles at locations β; the transfer function can be written as

$$H(Z) = \frac{(Z-\alpha_0)(Z-\alpha_1)\cdots(Z-\alpha_{m-1})}{(Z-\beta_0)(Z-\beta_1)\cdots(Z-\beta_{m-1})}$$

The design choice for practical implementation is the cascade form

$$H(Z) = \prod \frac{b_{k0} + b_{k1}Z^{-1} + b_{k2}Z^{-2}}{1 + a_{k1}Z^{-1} + a_{k2}Z^{-2}}$$

or the parallel form

Strategy

If the transfer function has m poles and zeros, with zeros at locations α and poles at locations β; the transfer function can be written as

$$H(Z) = \frac{(Z-\alpha_0)(Z-\alpha_1)\cdots(Z-\alpha_{m-1})}{(Z-\beta_0)(Z-\beta_1)\cdots(Z-\beta_{m-1})}$$

The design choice for practical implementation is the cascade form

$$H(Z) = \prod \frac{b_{k0} + b_{k1}Z^{-1} + b_{k2}Z^{-2}}{1 + a_{k1}Z^{-1} + a_{k2}Z^{-2}}$$

or the parallel form

$$H(Z) = \sum \frac{c_{k0} + c_{k1}Z^{-1}}{1 + a_{k1}Z^{-1} + a_{k2}Z^{-2}}$$

Strategy

There are many software tools that can be used to select the best strategy. In practice, MATLAB® is normally used. **Remember there is no optimum method that works for all types of filters.**

Questions

Filters and the $\mathbb{Z}$ Transform

Determine the $\mathbb{Z}$ Transform of the following digital filters; and determine the transfer function of the filters in the frequency domain.

1. $y_n = x_n + 2x_{n-1} + 2x_{n-3} + x_{n-4}$
2. $y_n + y_{n-1} = x_n + 2x_{n-1} + 2x_{n-3} + x_{n-4}$

Sampling and $\mathbb{Z}$ Transform

Suppose a digital system has at its input (before sampling)

$$25\cos(2\pi 10 t)$$

What angle θ in the Z domain would the signal above occur at if the signal is sampled at

1. 100 Hz
2. 50 Hz
3. 25 Hz
4. 12 Hz

Given two Digital Filters with transfer functions

$$\frac{Y}{X} = Z^2 + 2Z + 3$$

$$\frac{Y}{X} = Z^{-2} + 2Z^{-1} + 3$$

Determine the *difference equation*[3]. Sketch the structures of these filters.

[3] The Difference Equation is the relationship between y_n and x_n

Normalised Filters

Given a digital filter with transfer function

$$1 + 2Z^{-1} + 3Z^{-2} + 2Z^{-3} + Z^{-4}$$

Determine the magnitude and phase response of the filter at a normalised frequency[4] of 0.25

4 $\frac{f}{Fs}$

A Digital Filter has transfer function

$$\frac{Y}{X} = Z^{-1} - 1$$

Given this is used to filter a signal

$$25\cos(2\pi 10t)$$

which is sampled at 50 Hz; **calculate**

1. The value of Z on the unit circle that represents this frequency.
2. The magnitude of the transfer function at this particular Z.
3. The phase of the transfer function at this particular value of Z.
4. Determine the group delay (in number of samples) at this Z.